PROJECT REVIEW 2

Assessment 9-10 Marks

1. Project Details

Project Title	AuxFlow: Static Analysis for Multi-Pass Document Compilation
Student Name(s)	Upayan Mazumder
Register Number(s)	24BDS0367
Programme / Branch	B.Tech CSE (Data Science)
Project Guide	Banupriya R
Review Date	[Insert Review 2 scheduled date]

2. Project Overview

Brief introduction to the project:

AuxFlow is a compiler-based static analysis tool for LaTeX documents that use multi-pass compilation. It builds a dedicated compiler front end (lexer, parser, AST, symbol tables) and an intermediate representation called the Auxiliary Reference Graph (ARG), then performs graph-based dataflow analysis to classify a document's cross-reference dependency structure before any LaTeX engine runs.

Problem statement and the need for the project:

LaTeX documents containing cross-references, tables of contents, bibliographies, and similar auxiliary state require multiple compilation passes to converge to a stable output. Authors currently discover the correct number of passes by trial and error, or by running a fixed schedule (e.g. always three passes) that can either waste compute on documents that converge sooner or silently under-compile documents that need more passes, including rare documents that do not converge at all (infinite page growth or page-number oscillation). No existing tool reasons about this dependency structure statically, before executing the engine.

Objectives of the project:

- Build a complete compiler front end (lexer, parser, AST, symbol tables, semantic checks) for a declared, well-specified subset of the LaTeX language.

- Lower the resolved AST into an instance-level Intermediate Representation (the Auxiliary Reference Graph) that models cross-reference, table-of-contents, bibliography, and counter dependencies as typed, lagged graph edges.
- Apply static graph analysis (strongly connected components, taint propagation) to classify each document into one of four convergence states: CERTIFIED, BENIGN_CYCLIC, AMPLIFYING, or UNKNOWN.
- Derive a safe, minimal build plan (predicted engine pass count) for certified documents, and pair it with runtime verification that can confirm or correct the static prediction.
- Evaluate the approach empirically against a benchmark corpus and against existing shipping tools (latexmk, latexrun, Tectonic).

Proposed solution and its expected outcome:

AuxFlow proposes treating the LaTeX convergence problem as a compiler problem rather than a trial-and-error execution problem: parse the source once, build a dependency graph of every cross-reference-bearing construct, and decide statically whether the document's auxiliary state is guaranteed to stabilise, and in how many passes. The expected outcome is a tool that classifies a document and (when safe) predicts an exact engine-pass count in milliseconds, with zero LaTeX engine executions, while degrading conservatively (reporting UNKNOWN rather than guessing) for constructs outside its declared subset.

3. Implementation / Prototype Progress (75/100%) *(5 Marks)*

Overall implementation status:

As of Review 2, Phases 1 through 7 of the 16-phase implementation plan are complete and verified: the full compiler front end (lexical analysis, parsing, AST construction, symbol tables, semantic analysis) and the Intermediate Representation (the Auxiliary Reference Graph) are implemented, tested, and committed. This is approximately 44% of the total phase count and covers the entire input-to-IR pipeline that every later analysis phase depends on. The remaining phases (graph analysis, classification, build planning, runtime verification, and empirical evaluation) are scoped for Review 3.

Modules completed and their functionalities:

- **Lexer (auxflow.lexer):** Single-pass, table-driven scanner implementing all sixteen TeX category codes (catcodes). Recognises comments, escape sequences, control words/symbols, and delimiter-driven \verb verbatim regions. Verified with a token round-trip property test (reconstructed tokens reproduce the exact source bytes).
- **Parser (auxflow.parser):** Hand-written recursive-descent parser with one token of lookahead. Argument parsing (mandatory/optional groups) is driven by a declared macro-signature table. Produces one AST per file over sixteen node kinds, each carrying a

mandatory source span. Malformed input (unterminated groups, missing arguments) is recorded as a diagnostic rather than raising an exception, so the parser never crashes on real-world input.

- **AST and inclusion resolution (auxflow.ast):** Assembles per-file ASTs into a multi-file forest and builds the file-inclusion graph from \input/\include edges, with depth-first-search cycle detection for circular inclusion.
- **Symbol tables (auxflow.semantic):** Six symbol tables (LabelTable, RefTable, CiteTable, PackageTable, FileTable, CounterTable) populated by a two-pass declare-then-resolve traversal, so forward references and cross-file references both resolve correctly.
- **Semantic analysis (auxflow.semantic.diagnostics):** Zero-engine-execution checks: undefined references, undefined citations, duplicate labels, unresolvable file inclusions, missing graphics files, inclusion cycles, and use of an unmodelled (opaque) package.
- **Intermediate Representation (auxflow.ir):** The Auxiliary Reference Graph (ARG): one fact instance per modelled construct (label, reference, table-of-contents entry, citation, counter read), each with a mandatory source span, connected by typed, lagged dependency edges. List-producing facts use an exists/value split so that ordinary documents remain acyclic by construction. Serialises to JSON and to Graphviz DOT.
- **Command-line interface (auxflow.cli):** Five working subcommands exposing every completed phase: auxflow tokens, auxflow ast, auxflow symbols, auxflow check, auxflow graph.

Technologies, tools, programming languages, and platforms used:

- Language: Python 3.12+ (tested through 3.14), strict static typing enforced with mypy.
- Core implementation: zero third-party runtime dependencies. The lexer, parser, symbol tables, and IR are entirely hand-written, including the eventual Tarjan strongly connected components algorithm used in later phases.
- Testing: pytest with property-based testing via Hypothesis for round-trip and structural invariants.
- Code quality: ruff (linting and formatting), mypy --strict (type checking).
- Version control and CI: Git and GitHub, with GitHub Actions running lint, format-check, type-check, and the full test suite on every change.
- Target input language: a declared, documented subset of LaTeX2e.

System architecture / workflow:

The implemented pipeline (Phases 1-7, left to right): Source bytes --> Lexer (catcode-aware tokens) --> Parser (recursive descent) --> AST (per file) --> Multi-file Forest (inclusion graph) --> Symbol Tables (declare, then resolve) --> Semantic Diagnostics (zero-engine checks) --> IR Lowering --> Auxiliary Reference Graph (the Intermediate Representation). Every stage up to and including the IR runs with no LaTeX engine installed. Phases 8-16 (not yet started) will consume this ARG for graph analysis, classification, build planning, and runtime verification.

Screenshots of the implemented modules or prototype:

[Insert terminal screenshots here: output of `auxflow tokens`, `auxflow ast`, `auxflow symbols`, `auxflow check`, and `auxflow graph` run against a sample document, per the companion viva guide docs/review2-viva.md.]

Results obtained so far:

- Full front-end and IR pipeline (Phases 1-7) implemented and passing its test suite.
- ~3,900 lines of implementation code and ~4,100 lines of test code.
- All code passes strict mypy type checking with zero errors and zero ruff lint/format issues.
- Continuous integration (GitHub Actions) runs on every commit across Python 3.12 and 3.13.
- The parser correctly recovers from malformed input (unterminated groups, missing arguments) without raising an exception, verified by dedicated error-recovery tests.
- The IR's exists/value split was verified against hand-worked examples to keep ordinary documents (e.g. one containing only a table of contents) acyclic.

Remaining work required to complete the project:

- Phases 8-9: Strongly-connected-component (Tarjan) analysis and taint propagation to detect amplifying feedback.
- Phases 10-11: Stable-pass fixpoint derivation and the four-state document classifier (CERTIFIED / BENIGN_CYCLIC / AMPLIFYING / UNKNOWN).
- Phase 12: Build planner producing a safe, optimised engine-run schedule.
- Phase 13: Runtime verification executor that confirms or corrects static predictions against the real LaTeX engine.
- Phases 14-16: A fifteen-member benchmark corpus, a measurement harness comparing seven baselines, and the final empirical evaluation report.

4. Novelty *(2 Marks)*

What is new or different about the proposed project?

AuxFlow applies standard compiler-construction techniques (lexing, parsing, symbol tables, an intermediate representation, and graph-based dataflow analysis) to a problem domain (multi-pass LaTeX compilation convergence) that is conventionally handled only dynamically, by repeatedly re-running the engine. Building an instance-level dependency graph with a deliberate exists/value split to keep ordinary documents acyclic is the central structural idea.

How does the project differ from existing solutions?

Existing tools (plain LaTeX, latexmk, latexrun, Tectonic) all decide whether another compilation pass is needed dynamically: by re-running the engine and inspecting warnings or comparing auxiliary-file checksums after each run. None of them reason about the dependency structure before execution. AuxFlow instead performs this reasoning statically, from source, before any engine pass, classifying the document

What specific improvement, feature, or approach does it introduce?

- A declared, explicit LaTeX language subset with a documented default-deny rule: anything outside the subset is conservatively marked UNKNOWN rather than silently assumed safe.
- The exists/value fact split in the Intermediate Representation, which prevents the most common document shape (a table of contents) from appearing as a false cycle.
- A four-state classification (rather than a binary converges/does-not-converge verdict) that distinguishes harmless value-only cycles from genuinely amplifying feedback.
- A static analysis that is explicitly paired with, not a replacement for, runtime verification; it does not claim certainty it cannot support.

5. Presentation and Q&A *(3 Marks)*

Clear explanation of the problem, methodology, and implementation:

Presented via the accompanying slide deck (presentation/review2-auxflow.pptx) and viva preparation guide (docs/review2-viva.md), covering: why this is a compiler project; the lexer's category-code model; the recursive-descent parser and error recovery; the two-pass symbol table resolution; and the Auxiliary Reference Graph, including a worked end-to-end trace from a source snippet through tokens, AST, symbol bindings, and final IR facts and edges.

Explanation of individual contributions:

This is a single-student project. Upayan Mazumder (24BDS0367) is solely responsible for the design and implementation of all completed modules: the lexer, parser, AST, inclusion-graph resolution, symbol tables, semantic diagnostics, the Intermediate Representation, the command-line interface, the full test suite, and the continuous integration configuration.